How to Design an Android Interface

Mohammed Said BELAID
said.belaid@univ-usto.dz

22/02/2024
@USTO → FMI → Dept Info → L3 SI

Outline

1 Introduction

2 Views (UI Components)

3 Layouts

4 Best practices

5 Jetpack Compose

Objective

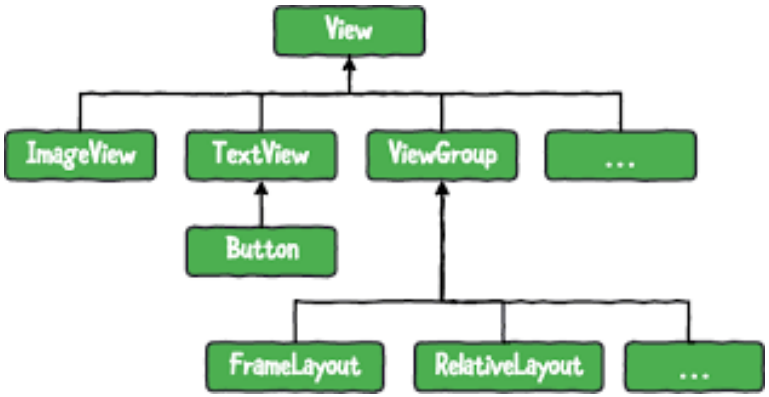
- Implement a user interface design for Android application using `XML`
- The most common visual components
- Understand the structure of an `XML` layout file

Different ways to create an Android interface

- `XML` Views using code
- Drag & Drop using design, not recommended!
- Jetpack Compose using `Kotlin`

Definition

- The basic building block for user interface components
- The main class of UI components (buttons, images, text fields)

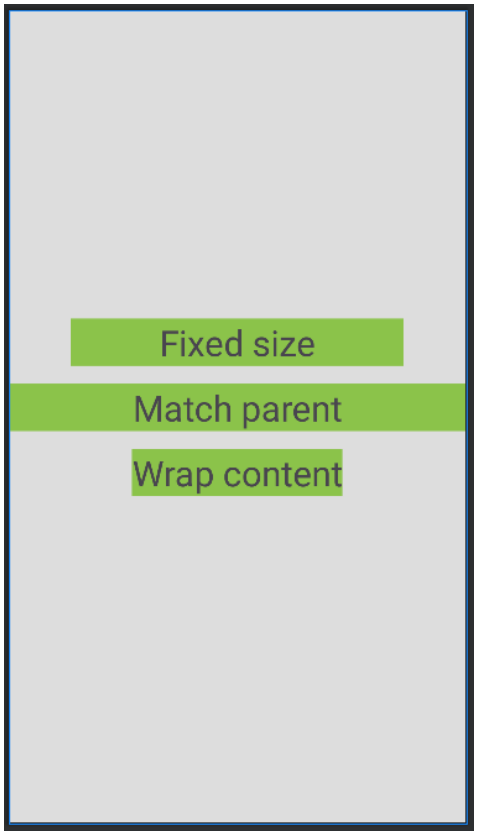


Common attributes (Dimensions)

- Each UI component should have both `width` and `height` dimensions set.
- Possible values:
  - a fixed size (`300dp`)
  - fill the size of the parent `match_parent`
  - adjust size based on the content `wrap_content`

Common attributes (Dimensions)

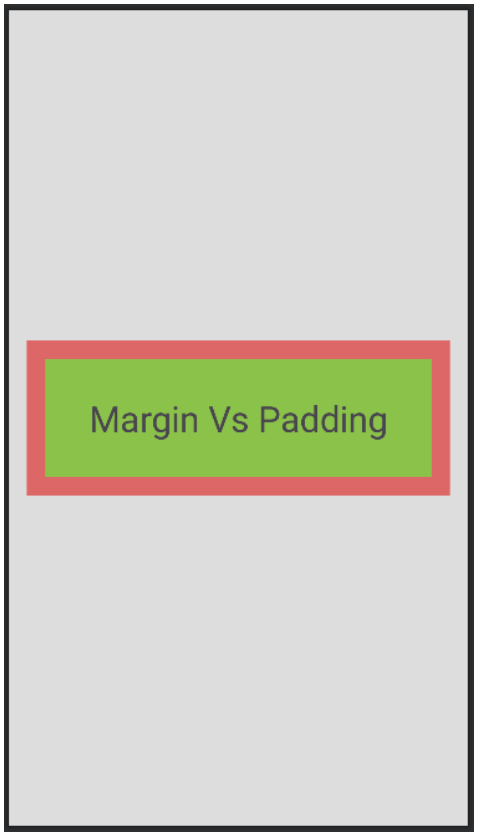
```
<TextView
    android:layout_width="300dp"
    .../>
<TextView
    android:layout_width="
        match_parent"
    .../>
<TextView
    android:layout_width="
        wrap_content"
    .../>
```



Common attributes (Spacing)

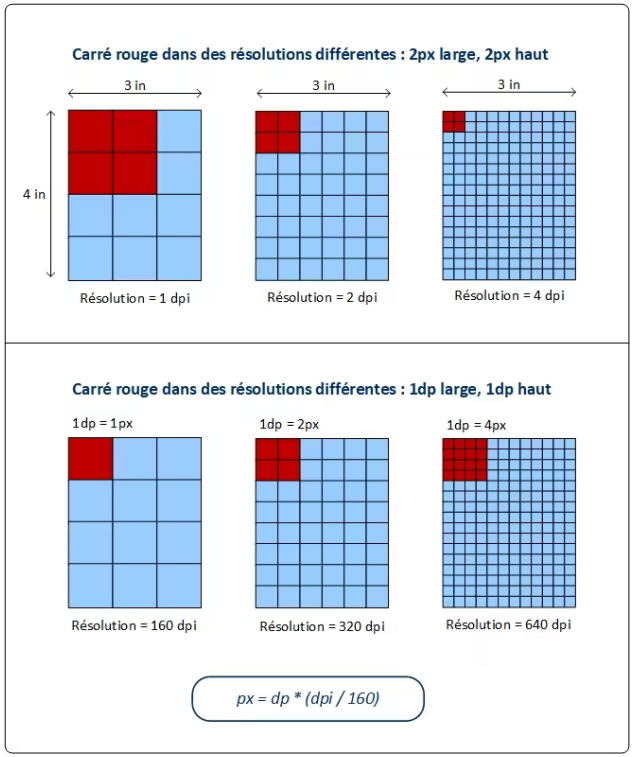
- Similar to HTML, we can set the box margin and padding.
- Margin: the space outside the boundary of a View.
- Padding: the space inside the boundary of a View.
- Can be set individually for each side or globally

```
<TextView
    android:layout_margin="16dp"
    android:padding="32dp"
    .../>
```



Dimension Units

- Pixel (px): the absolute pixels on the device screen, not recommended!
- Density independent Pixels (dp): a consistent size across different screen densities
- Scale-independent Pixels (sp): used for text sizes, scales with the user's preferred text size settings



<TextView/>

A user interface element that displays text to the user.

```
<TextView
    ...
    android:text="This is a
        simple TextView"/>
```

<TextView/>

```
<TextView
    ...
    android:textSize="32sp"
    android:text="This is a
        simple TextView"/>
```

<TextView/>

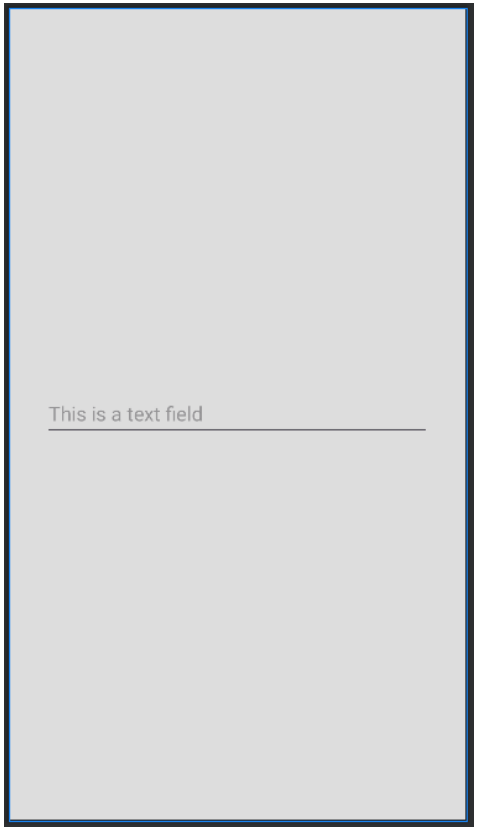
```
<TextView
    ...
    android:textColor="#3F51B5"
    android:text="This is a
        simple TextView"/>
```

<Button/>

```
<Button
    android:layout_width="
        wrap_content"
    android:layout_height="
        wrap_content"
    android:text="This is a
        Button"/>
```

<EditText/>

```
<EditText
    android:layout_width="
        match_parent"
    android:layout_height="
        wrap_content"
    android:inputType="number"
    android:hint="This is a text
         field" />
```



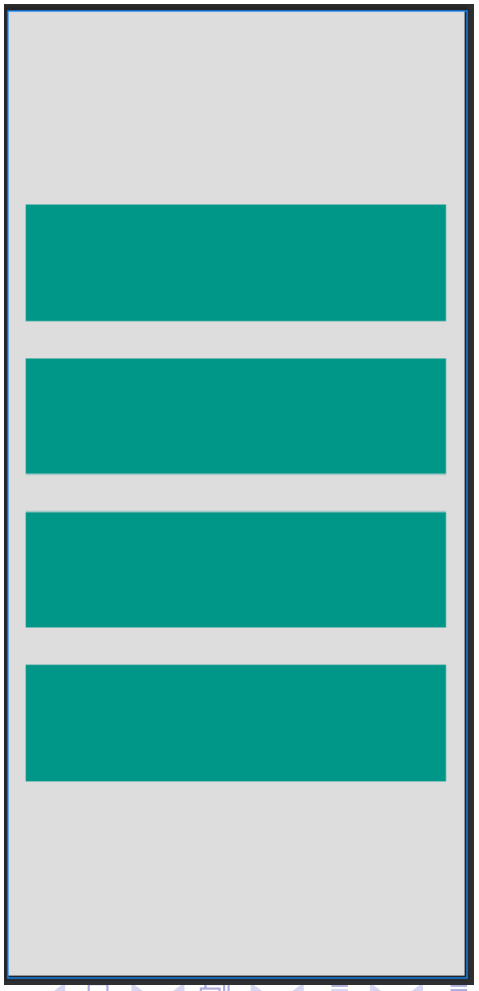
Layouts and Containers

- define how the UI components are organized and positioned on the screen
- the structure and arrangement of Views within the user interface
- provide a foundation to build responsive user interfaces
- Examples: `LinearLayout`, `RelativeLayout`, `ConstraintsLayout`

<LinearLayout>

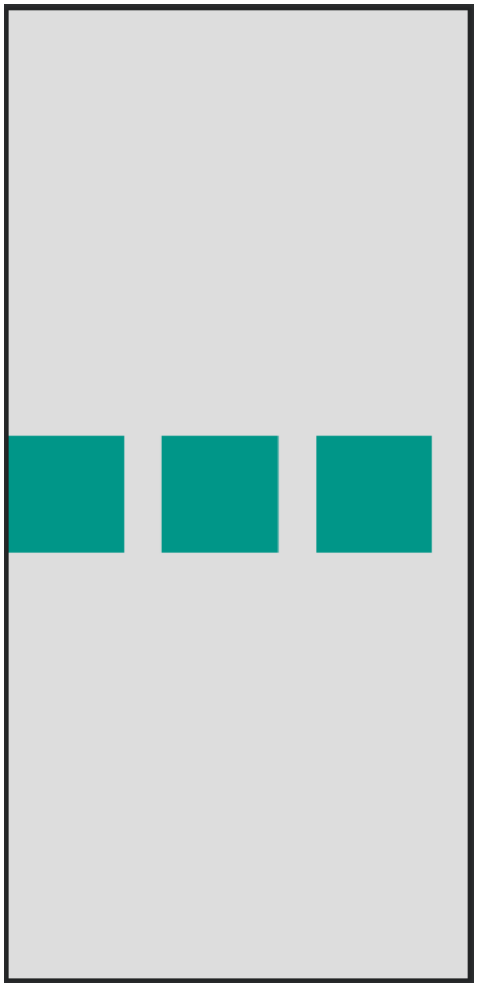
Arranges the child Views linearly either horizontally or vertically.

```
<LinearLayout
    ...
    android:orientation="
        vertical">
    <TextView .../>
    <TextView .../>
    <TextView .../>
</LinearLayout>
```



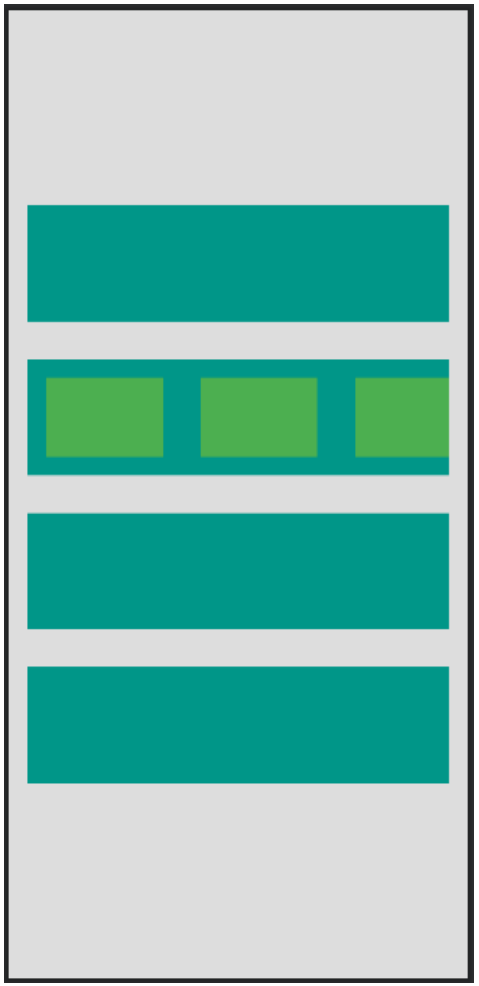
<LinearLayout>

```
<LinearLayout
    ...
    android:orientation="
        horizontal">
    <TextView .../>
    <TextView .../>
    <TextView .../>
</LinearLayout>
```



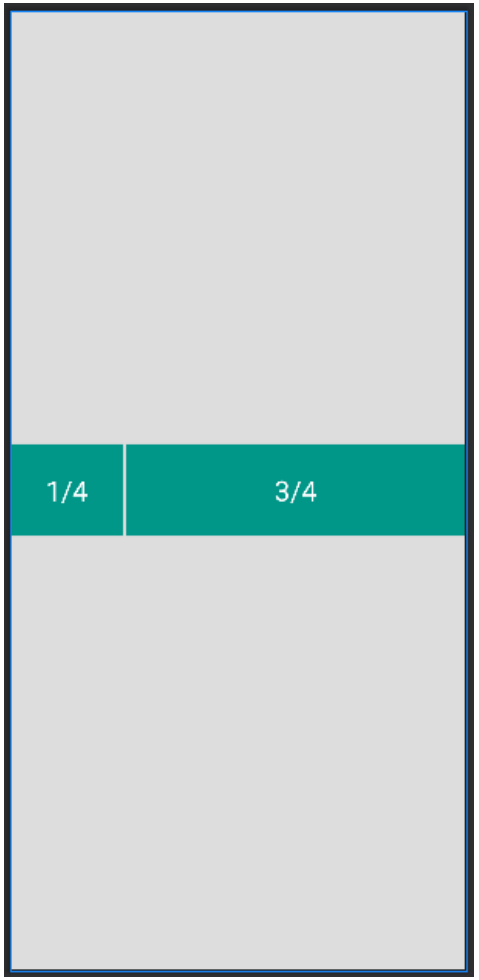
Nested <LinearLayout>

```
<LinearLayout
    ...
    android:orientation="
        vertical">
  <TextView .../>
  <LinearLayout
      ...
      android:orientation="
          horizontal">
      <TextView .../>
      <TextView .../>
  </LinearLayout>
  <TextView .../>
  <TextView .../>
</LinearLayout>
```



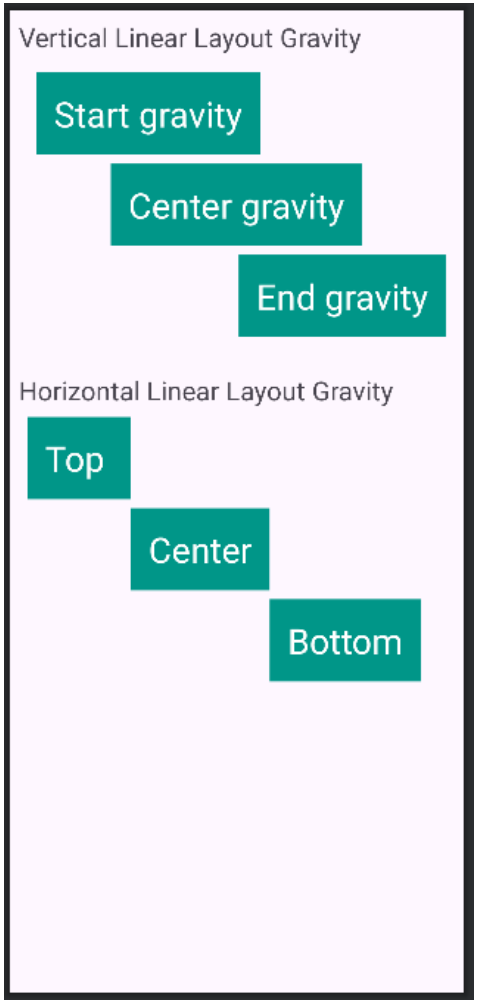
Weighted <LinearLayout>

```
<LinearLayout
    ...
    android:orientation="
        horizontal"
    android:weightSum="4">
    <TextView
  android:layout_weight="1"
  .../>
    <TextView
  android:layout_weight="3"
  .../>
</LinearLayout>
```



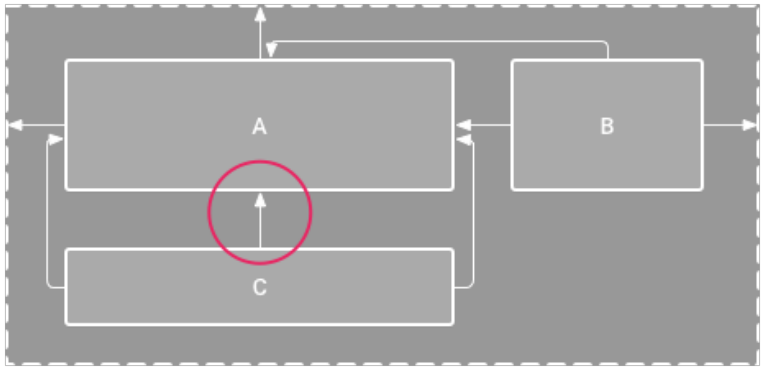
Gravity

```
<LinearLayout
    ...
    android:orientation="
        horizontal">
    <TextView
  android:layout_gravity="center
      "
  .../>
</LinearLayout>
```



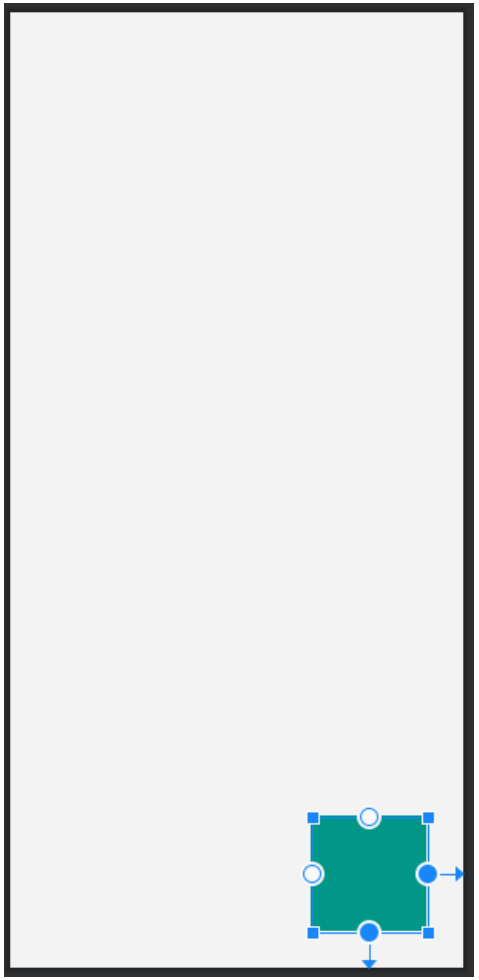
ConstraintsLayout

- positioning and sizing of child Views relative to each other or to the parent using constraints
- highly flexible and efficient for creating complex UI designs



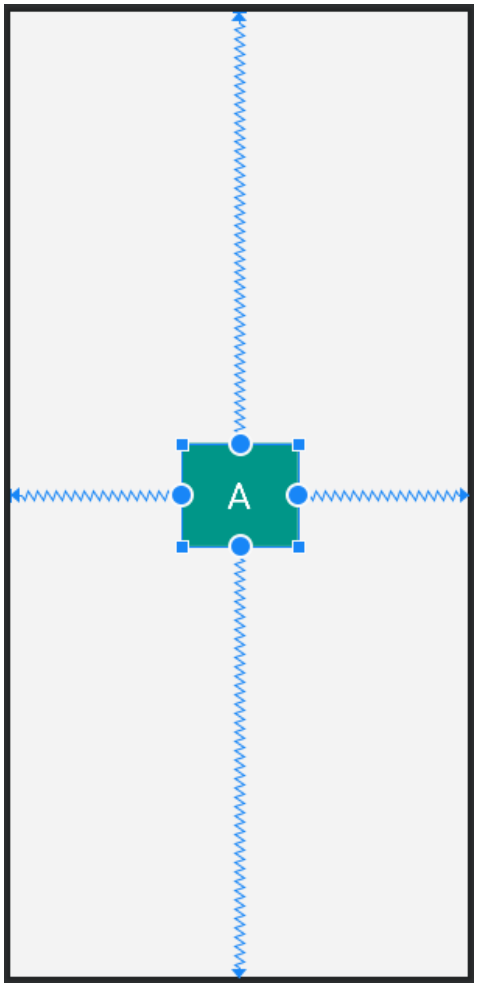
ConstraintsLayout relative to parent

```
<ConstraintLayout>
  <View
  app:
      layout_constraintBottom_toBottomOf
      ="parent"
  app:
      layout_constraint End_toEndOf
      ="parent"/>
</ConstraintLayout>
```



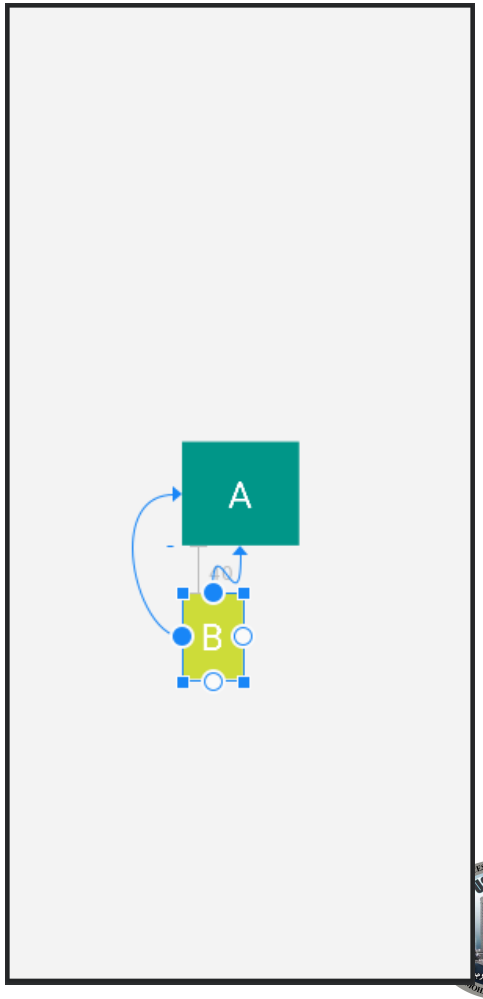
ConstraintsLayout relative to parent

```
<ConstraintLayout>
  <TextView
  app:
      layout_constraintTop_toTopOf
      ="parent"
  app:
      layout_constraintBottom_toBottomOf
      ="parent"
  app:
      layout_constraintStart_toStartOf
      ="parent"
  app:
      layout_constraintEnd_toEndOf
      ="parent"/>
</ConstraintLayout>
```



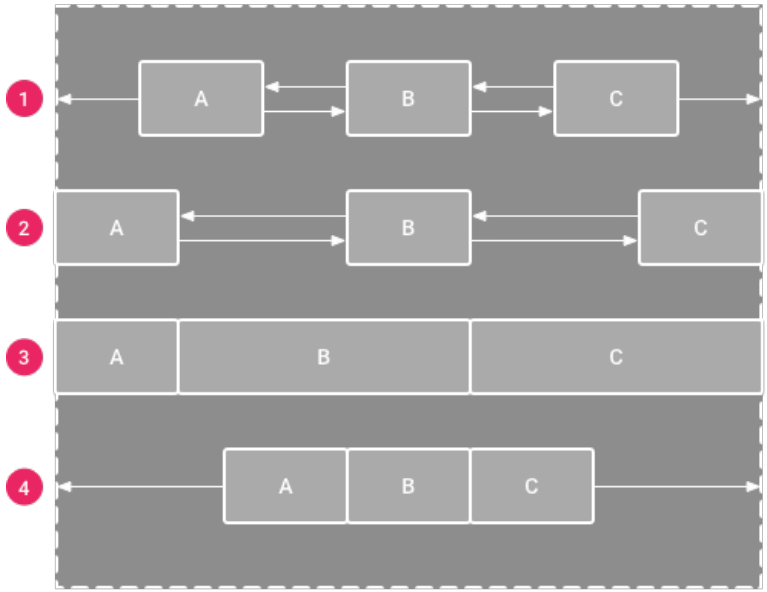
ConstraintsLayout relative to component

```
<ConstraintLayout>
  <TextView
  android:text="A"
  android:id="@+id/aView"
  layout_constraintTop_toTopOf="parent"
  layout_constraintBottom_toBottomOf="parent"
  layout_constraintStart_toStartOf="parent"
  layout_constraintEnd_toEndOf="parent"/>
  <TextView
  android:text="B"
  layout_constraintStart_toStartOf="@id/aView
      "
  layout_constraintTop_toBottomOf="@id/aView"
      />
</ConstraintLayout>
```



ConstraintsLayout chains

- `spread`: distribute margin
- `spread_inside`: same, first and last are excluded
- `weight`: by percentage
- `packed`: packed together, with margin on start and end



Other Containers

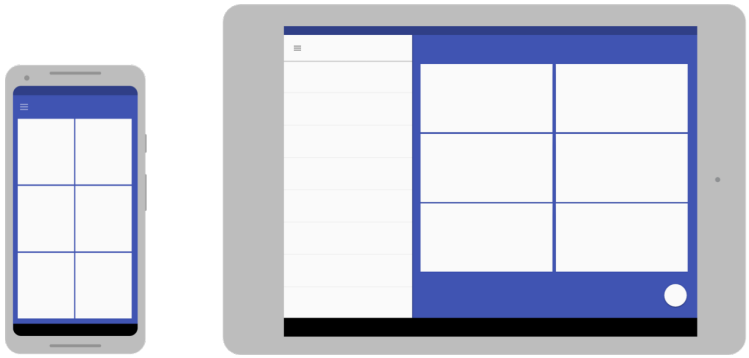
- `RelativeLayout`: similar to `ConstraintsLayout`, less efficient
- `FrameLayout`: contains one single child
- `GridLayout`: arranges child views in a grid format, with rows and columns.
- `CoordinateLayout`: specifically designed to coordinate animations and transitions
- `ScrollView`: allows its child views to be scrolled vertically or horizontally

Best practices

- Avoid using hard-coded strings, use `strings.xml` for all the static strings
  - strings can be reused in multiple layouts
  - translation will be easier, just create another file the targeted language
- Use `start` and `end` instead of `left` and `right`
  - ensure a correct positioning for languages written from right to left (rtl)
- Use `colors.xml`, `dimens.xml` to store colors and shared dimensions
- Use styles and themes
- Implement for different screen sizes

Adaptive Layout

- *Responsive Design*: a single version of interface that adapts dynamically to the dimensions of the user's screen.
- *Adaptive Design*: create multiple versions of the same screen for specific device categories (phone, tablet, large tablets . . .)



Adaptive Layout

Use specific folder for the same interface, for example `layout-land` for landscape screens.

```
res/layout/main_activity.xml                # For phones
res/layout-land/main_activity.xml           # For phones in
res/layout-sw600dp/main_activity.xml        # For 7" tablets
res/layout-sw600dp-land/main_activity.xml   # tablets in lan
```

Jetpack Compose

- a modern UI toolkit for Android app development
- using a declarative programming paradigm
- Supports only `Kotlin`

Jetpack Compose

```
@Composable
fun Test() {
  Column {
    Row {
      Image(...)
      Column {
  Text(...)
  Text(...)
      }
    }
  }
}
```

Jetpack Compose

For more:
https://developer.android.com/jetpack/compose/tutorial